



← Back to student guides

 DVC

Data versioning

3 levels

113 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

CODE

POINTER

CACHE

REMOTE



Quick Start

 Open PDF

A one-page starter for this tool. Scroll through it here, or open it full screen.





Why This Matters

- Consistency**
Stable file paths in code forever — no more data/20190922/labels_v7_final to chase down.
- Efficient storage**
Content-addressed caching means identical files are never duplicated, even across many versions.
- Cost control**
Cheap object storage (S3/GCS) replaces paying for Git-hosting storage limits or LFS bandwidth.
- Auditability**
Every data change is a reviewable Git commit — or even a pull request — with a full, immutable history.

MLOps MENA Community

19

02

Smart Pipelines with dvc.yaml

Build a pipeline once — let dvc repro decide what actually needs to rerun.

MLOps MENA Community

20

IDEA 6 — SMART PIPELINES

A Pipeline Is a Dependency Graph

- Stages are the building blocks**
Each stage is one command, with declared dependencies and declared outputs.
- DVC connects them automatically**
One stage's output becomes the next stage's dependency — no manual wiring needed.
- The result is a DAG**
A Directed Acyclic Graph — dvc dag prints (or exports) it so the whole pipeline shape is visible at a glance.
- Order in the file doesn't matter**
DVC works out execution order from the dependency graph itself, not from the order stages are written in dvc.yaml.

MLOps MENA Community

21

IDEA 6 — SMART PIPELINES

Anatomy of dvc.yaml

```
stages:
  prepare:
    cmd: python src/prepare.py data/data.xml
    deps: [data/data.xml, src/prepare.py]
    params: [prepare.seed, prepare.split]
    outs: [data/prepared]

  featurize:
    cmd: python src/featurization.py data/prepared data/features
    deps: [data/prepared, src/featurization.py]
    params: [featurize.nm_features, featurize.ngrams]
    outs: [data/features]

  train:
    cmd: python src/train.py data/features
    deps: [data/features, src/train.py]
    outs: [model.pkl]
```

prepare always runs before featurize, because data/prepared is prepare's output and featurize's dependency.

...

22

Three Kinds of Dependencies



18

sections

42

examples

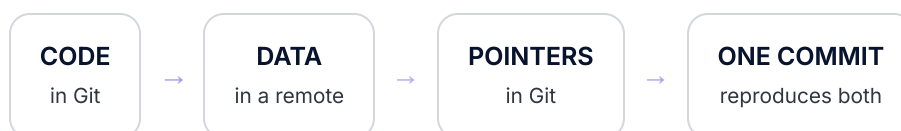
This is part one of three. It covers **everything you need to do real work with DVC**, not a teaser. By the end you can version a dataset, reproduce a pipeline, share data with a colleague through remote storage, compare experiment metrics, and answer "which data produced this model?" with a commit hash. Mid-level and Senior take the same topics further; nothing here is thrown away.

Each section ends with a **Try it** task. Do them as you go. They take a few minutes each, and these ideas only stick once you have watched [dvc repro](#) skip a stage it did not need to run.

The problem DVC solves: code in Git, data elsewhere

Git is excellent at versioning text and hopeless at versioning data. Commit a 2 GB training set and you have permanently added 2 GB to every clone of that repository, forever, because Git stores full history. Change one row and you add another 2 GB.

So teams do the obvious thing and keep data out of Git. Then the real problem starts: the model in production was trained on [something](#), and nobody can say what. The notebook says [data/train_v3_final_fixed.csv](#), that file is on a laptop that was reimaged, and the person who made it left.



DVC's central trick is to store a small text pointer in Git and the actual bytes somewhere cheap. The pointer file is a few hundred bytes and contains a content hash. Git versions the



That single idea gives you four things, and they are the reason DVC exists rather than a bespoke script:



Data tied to commits

Every commit records which dataset and which model it used. "Which data produced this?" becomes a lookup.



Reproducible pipelines

Declare stages and dependencies once. DVC reruns only what changed.



Shared storage

Push data to S3, GCS, Azure, or an SSH box. A colleague clones and pulls, and has your exact files.



Comparable experiments

Metrics and parameters are versioned alongside the code, so comparing two runs is a command, not archaeology.

You need little to follow along: Python, Git, and a folder. A tiny CSV and a five-line training script are the best place to start, because a broken pipeline costs you nothing.

► TRY IT

1. Find a project of yours with data in it. Run `git count-objects -vH` and note the `size-pack`.
2. Look for files matching `*_final*`, `*_v2*`, `*_backup*`, or a date in the name. Count them.
3. Pick the newest model artifact you have and try to name, precisely, the exact file that trained it.

Expected result – either a repository bloated by data, or a folder full of manually versioned filenames, usually both. That third question is the one DVC turns from an archaeology exercise into a command.



DVC is a Python package, and it sits **on top of** Git rather than replacing it.

BASH

Copy

```
pip install "dvc[s3]"          # or dvc[gs], dvc[azure], dvc[ssh], dvc[all]
dvc --version
```

The bracketed extra pulls in the driver for your remote storage. Plain `pip install dvc` works fine for local experimentation and then fails with a confusing "URL is not supported" the moment you configure an S3 remote, so install the extra you need up front.

BASH

Copy

```
git init                      # DVC needs a Git repository
dvc init
git status
```

`dvc init` creates a small amount of scaffolding and stages it for you:

Path	Purpose
<code>.dvc/config</code>	Your project's DVC configuration: remotes, cache settings. Committed
<code>.dvc/.gitignore</code>	Keeps DVC's internal cache and temp files out of Git
<code>.dvcignore</code>	Optional; excludes paths from DVC's own scanning, like <code>.gitignore</code>
<code>.dvc/cache/</code>	Where content lands locally, addressed by hash. Not committed

BASH

Copy

```
git commit -m "Initialise DVC"
```

DVC does not replace Git. It needs it



almost always yes for anything DVC creates that is not inside `.dvc/cache`, and DVC tells you the exact command to run.

► TRY IT

1. Create a fresh folder, run `git init`, then `dvc init`.
2. Run `git status` and read what DVC staged for you.
3. Open `.dvc/.gitignore` and note that `/cache` is listed.
4. Commit, then run `dvc doctor` and read the output.

Expected result – a clean initialisation with three or four staged files, and a `.gitignore` that already excludes the cache. `dvc doctor` prints your platform, version, and which remote drivers are available, worth knowing about now because it is the first thing to run when something behaves oddly.

Your first tracked dataset, in four commands

This is the core loop. Four commands, and afterwards your data is versioned.

BASH

Copy

```
mkdir -p data
# put a file in data/raw.csv – any size, even 10 rows

dvc add data/raw.csv
git add data/raw.csv.dvc data/.gitignore
git commit -m "Track the raw dataset"
```

Now look at what happened, because this is the whole mental model:

BASH

Copy



data/raw.csv.dvc

Copy

```
outs:
- md5: 8f2c4b19e0a7d3f1c6b8a2e4d7091f3b
  size: 1048576
  hash: md5
  path: raw.csv
```

The `.dvc` file holds a hash, a size, and a path. **The file itself is not in Git.** `data/.gitignore` now contains `/raw.csv`, which DVC wrote for you, so Git will never try to store it.

BASH

Copy

```
cat data/.gitignore      # /raw.csv
git status                # clean; the CSV is ignored
```

The bytes went into DVC's cache, filed under that hash:

BASH

Copy

```
ls .dvc/cache/files/md5/8f/2c4b19e0a7d3f1c6b8a2e4d7091f3b
```

data/raw.csv
your working copy



.dvc/cache
filed by hash



raw.csv.dvc
the pointer, in Git

Command	Does
<code>dvc add PATH</code>	Hash the file, move it to the cache, write a <code>.dvc</code> pointer, add to <code>.gitignore</code>
<code>dvc status</code>	Compare working files against the pointers
<code>dvc checkout</code>	Restore working files from the cache to match the pointers
<code>dvc remove PATH.dvc</code>	Stop tracking; removes the pointer and the <code>.gitignore</code> entry



missing or edited `.gitignore` entry. Committing the file to Git as well as DVC defeats the whole purpose and doubles your storage. `git status` being clean after `dvc add` is the signal that it worked.

► TRY IT

1. Create `data/raw.csv` with a handful of rows and run `dvc add data/raw.csv`.
2. Read the three things DVC changed: the new `.dvc` file, the new `.gitignore` entry, and the cache directory.
3. Commit the pointer and confirm `git status` is clean.
4. Now check the sizes: `du -h data/raw.csv .dvc/cache` and `wc -c data/raw.csv.dvc`.

Expected result – a few hundred bytes in Git standing in for however large your file is. Seeing those two numbers side by side is the moment the design clicks.

Inside the cache: content addressing and links

Understanding this saves you from a whole category of confusion later, and it takes two minutes.

DVC's cache is **content-addressable**: a file's location is derived from the hash of its contents. Two identical files, whatever they are called, occupy one cache entry. Change one byte and you get a different hash and a new entry.

BASH

Copy

```
dvc add data/raw.csv          # hash A
echo "one more row" >> data/raw.csv
dvc add data/raw.csv          # hash B – a new entry; A is still there
```



and also why the cache grows.

By default DVC does not copy the file into your workspace, it **links** it from the cache, so you are not storing two copies:

Link type	Behaviour	Default on
<code>reflink</code>	Copy-on-write; fast, safe, and space-efficient	APFS, Btrfs, XFS where supported
<code>copy</code>	A real second copy. Safe, uses double the space	The fallback everywhere
<code>hardlink</code> / <code>symlink</code>	One inode, no extra space, but editing in place corrupts the cache	Opt-in

BASH

Copy

```
dvc config cache.type reflink,copy      # try reflink, fall back to a copy
dvc config core.check_link_support true
```

Never edit a file in place under hardlink or symlink caching

With those link types your working file and the cache entry are the same bytes on disk, so editing the file silently rewrites cached history for every version that shared it. If you enable them, treat working files as read-only and always regenerate rather than edit.

`reflink,copy` is the safe default and what you want unless you have measured a reason otherwise.

► TRY IT

1. Run `dvc add` on a file, note the md5 in the `.dvc` file, then append one line and `dvc add` again.
2. Compare the two hashes, then look in `.dvc/cache/files/md5/` and confirm both entries exist.
3. Copy the file to a second name and `dvc add` that too. Check whether the cache grew.



Expected result – two cache entries for two versions, but only one for two identical files under different names. That deduplication is free, and it is why adding the same dataset in three projects on one machine costs you one copy with a shared cache.

Going back in time: two commands, in order

This is the payoff. Do it on purpose early so you trust it.

BASH

Copy

```
# Make a second version and commit it
echo "new rows" >> data/raw.csv
dvc add data/raw.csv
git commit -am "Dataset v2"

# Travel back
git checkout HEAD~1 data/raw.csv.dvc
dvc checkout data/raw.csv
head data/raw.csv                # v1 is back
```

Two commands, and the order matters. **git checkout** moves the pointer; **dvc checkout** makes the working file match it. Miss the second step and you have v1's pointer with v2's file on disk, which **dvc status** will tell you about immediately.

BASH

Copy

```
dvc status
# data/raw.csv.dvc:
#      changed outs:
#      modified:      data/raw.csv
```



- Instant, because pointers are tiny
- Leaves data files untouched

- Reads from the local cache
- Fails if the cache does not have that version, so then you need `dvc pull`

Install the Git hooks and stop thinking about it

`dvc install` adds Git hooks that run `dvc checkout` after `git checkout`, and warn you about unpushed data before `git push`. It removes the most common source of "why is my data the wrong version?" A forgotten second command.

► TRY IT

1. Create three versions of a file, committing after each `dvc add`.
2. Use `git log --oneline` to find the first commit, then `git checkout <sha> data/raw.csv.dvc`.
3. Run `dvc status`

Expected result – before

`dvc checkout` and read what it reports.

4. Run `dvc checkout`, confirm the file matches v1, then return to the latest with `git checkout HEAD data/raw.csv.dvc && dvc checkout`.
5. Now run `dvc install` and repeat step two, and notice you no longer need step four.

Expected result – the intermediate state where the pointer and the file disagree, which `dvc status` names precisely. Seeing that state once means you will recognise it instantly, and `dvc install` means you rarely have to.

Remote storage: where the bytes live

So far everything lives on your machine. A remote is where the bytes go so a colleague (or a CI runner, or a training box) can get them.



```
# Local folder: perfect for learning, and genuinely useful for a shared NAS
dvc remote add -d myremote /tmp/dvc-storage

# S3
dvc remote add -d storage s3://my-bucket/dvc-store

# Google Cloud Storage
dvc remote add -d storage gs://my-bucket/dvc-store

# Azure Blob
dvc remote add -d storage azure://my-container/dvc-store

# Any SSH box
dvc remote add -d storage ssh://user@host/srv/dvc-store

git add .dvc/config
git commit -m "Configure the default DVC remote"
```

The `-d` makes it the default, so `dvc push` and `dvc pull` need no arguments. The configuration lands in `.dvc/config`, which **is** committed. It contains the location, not the credentials.

BASH

Copy

```
dvc push          # upload cached content to the remote
dvc pull          # download what the current pointers need
dvc fetch         # download to the cache without touching the workspace
dvc status -c     # compare local cache against the remote
```

Those four are the whole workflow, and the mental model mirrors Git exactly:

VERSIONED IN GIT: SMALL, DIFFABLE, REVIEWABLE

Code & `dvc.yaml`

Scripts, stages, params

`*.dvc` · `dvc.lock`

Hash, size, path: a few hundred bytes



`.dvc/cache`

Content-addressed, local, gitignored

Remote

S3 · GCS · Azure · SSH

Hash addresses the bytes

`files/md5/8f/2c4b...`, the same layout in cache and remote

`dvc add` · `checkout`

Workspace ↔ cache

`dvc push` · `pull` · `fetch`

Cache ↔ remote

`dvc gc --cloud`

Deletes on the remote. Not recoverable

Read it as two hops: a pointer in Git names a hash, the cache holds that hash locally, and the remote holds it for everyone else. `git push` ships the pointer; `dvc push` ships the bytes. Do one without the other and a colleague gets a reference to data they cannot fetch.

Git	DVC	Moves
<code>git add</code>	<code>dvc add</code>	Working file → tracked
<code>git commit</code>	(commit the <code>.dvc</code> file)	Pointer → history
<code>git push</code>	<code>dvc push</code>	Cache → remote
<code>git pull</code>	<code>dvc pull</code>	Remote → cache → workspace
<code>git status</code>	<code>dvc status</code> / <code>dvc status -c</code>	What differs, locally / against the remote

Credentials do not belong in `.dvc/config`



`gitlab auth`, an SSH agent. If you must store something per-machine, DVC has a **local** config for exactly that, covered in the next section.

► TRY IT

1. Add a local-folder remote pointing at `/tmp/dvc-storage` and commit `.dvc/config`.
2. Run `dvc push`, then look inside `/tmp/dvc-storage` for the same hash-based layout as the cache.
3. Run `dvc status -c` and confirm everything is up to date.
4. Now simulate a colleague: `rm -rf .dvc/cache data/raw.csv`, then `dvc pull`.

Expected result – your file comes back from the remote after you deleted both it and the local cache. That is the moment DVC stops being an abstraction and starts being useful, and the remote's directory layout being identical to the cache is not a coincidence.

Two config files: committed versus local

There are two config files, and knowing which is which prevents both leaked secrets and "it works on my machine".

BASH

Copy

```
dvc remote add -d storage s3://my-bucket/dvc-store # → .dvc/config, committed
dvc remote modify --local storage access_key_id AKIA... # → .dvc/config.local, ignored
```

File	Committed	For
<code>.dvc/config</code>	Yes	Remote URLs, cache type, anything the whole team shares



<code>.dvc/config.local</code>	NO (gitignored)	Credentials, machine-specific paths, a personal cache directory
--------------------------------	------------------------	---

The pattern that follows from this is worth adopting immediately: **shared truth in `config`** , **personal reality in `config.local`** . A remote URL is shared truth. Your access key is personal reality. So is a cache on a different disk because your home directory is small.

BASH

Copy

```
# Useful local-only settings
dvc cache dir /mnt/big-disk/dvc-cache          # move the cache off a small root disk
dvc remote modify --local storage ssh_private_key_path ~/.ssh/id_dvc
dvc config --local core.jobs 8                 # more parallel transfers on a fast link
```

A read-only remote for people who should not push

You can define several remotes and give one to contributors who need to `pull` but must not `push` . Access control lives in the storage provider, not in DVC, but declaring the remote in `.dvc/config` means everyone at least agrees on where the data is.

► TRY IT

1. Run `dvc remote list` and then `cat .dvc/config` .
2. Set something local-only: `dvc config --local core.jobs 4` , then `cat .dvc/config.local` .
3. Run `git status` and confirm `config.local` is not listed.
4. Check `.dvc/.gitignore` and find the entry that makes that true.

Expected result – two files, one committed and one ignored, with the ignore rule already written for you. Knowing this split exists is what stops a credential ending up in your history six months from now.



Datasets are rarely one file. `dvc add` works on directories, and the behaviour is slightly different in a way worth knowing.

BASH

Copy

```
dvc add data/images/          # one .dvc file for the whole directory
cat data/images.dvc
```

data/images.dvc

Copy

```
outs:
- md5: 3a7f9c2e8b1d4506.dir    # note the .dir suffix
  size: 524288000
  nfiles: 12000
  hash: md5
  path: images
```

The `.dir` suffix marks this as a **directory hash**, the hash of a small JSON listing that maps every file in the tree to its own hash. Individual files are still deduplicated in the cache, so adding a directory where one image changed uploads exactly one new object.

Property	Single file	Directory
Pointer records	One hash	A <code>.dir</code> hash plus <code>nfiles</code>
Cache entries	One	One per file, plus the listing
Changing one file	New hash for the file	New <code>.dir</code> hash, one new file entry
<code>dvc status</code> granularity	The file	Counts of added, modified, deleted

Directories with hundreds of thousands of small files are slow

DVC hashes every file, so `dvc add` on a tree of 500,000 tiny images takes a long time and produces a large listing. If that is your situation, the answer is usually to package the



► TRY IT

1. Create `data/images/` with twenty small files and run `dvc add data/images` .
2. Read the `.dvc` file and note the `.dir` hash and `nfiles` .
3. Change one file, run `dvc status` , and read how it reports the difference.
4. `dvc add` again and `dvc push` , then count the objects that were uploaded.

Expected result – a single pointer for the whole tree, and a second push that uploads one object rather than twenty. That per-file deduplication inside a directory is what makes DVC practical for image datasets.

Pipelines: from `dvc add` to `dvc.yaml`

`dvc add` versions data somebody produced. A **pipeline** versions the `process` , which is what makes a result reproducible rather than recorded.

The idea is a dependency graph. Each stage declares what it needs and what it makes, and DVC reruns a stage only when one of its declared inputs changed.

`dvc.yaml`

Copy

```
stages:
  prepare:
    cmd: python src/prepare.py data/raw.csv data/prepared.csv
    deps:
      - src/prepare.py
      - data/raw.csv
    outs:
      - data/prepared.csv

  train:
    cmd: python src/train.py data/prepared.csv models/model.pkl
    deps:
```



- models/model.pkl

BASH

Copy

```
dvc repro # run whatever is out of date, in dependency order
git add dvc.yaml dvc.lock .gitignore
git commit -m "Add the prepare → train pipeline"
```

Four keys carry almost all of the meaning:

Key	Means
<code>cmd</code>	The command to run. Any executable: Python, R, a shell script, a binary
<code>deps</code>	Inputs. If any changes, this stage is out of date
<code>outs</code>	Outputs. DVC tracks these, caches them, and gitignores them for you
<code>params</code>	Values read from <code>params.yaml</code> , so a config change invalidates the stage

Notice what you no longer do: **you do not `dvc add` a pipeline output.** `outs` already tracks it. Running `dvc add` on something a stage produces is a common early mistake and DVC will refuse it, because two mechanisms would then own the same file.



Write `dvc.yaml` by hand

There is a `dvc stage add` command that generates it, and it is fine. But `dvc.yaml` is a short, readable file that you will edit far more often than you create, so learning to write it directly is faster within about ten minutes.

► TRY IT



3. Run `dvc repro` again with no changes and read the output.

4. Try to `dvc add` one of the outputs and read the error.

Expected result – the second run reports that everything is up to date and executes nothing, and the `dvc add` attempt is refused because the file is already a stage output. Those two behaviours are the pipeline model in miniature.

`dvc.yaml` is what you wrote. `dvc.lock` is what happened, and it is the file that makes reproducibility real.

dvc.lock

Copy

```
schema: '2.0'
stages:
  prepare:
    cmd: python src/prepare.py data/raw.csv data/prepared.csv
    deps:
      - path: data/raw.csv
        hash: md5
        md5: 8f2c4b19e0a7d3f1c6b8a2e4d7091f3b
        size: 1048576
      - path: src/prepare.py
        hash: md5
        md5: 1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d
        size: 842
    outs:
      - path: data/prepared.csv
        hash: md5
        md5: 9e8d7c6b5a4f3e2d1c0b9a8f7e6d5c4b
        size: 987654
```



compares them against the lock.

1 Read the graph

DVC builds the dependency order from `deps` and `outs` across all stages.

2 Hash the inputs

For each stage, hash every dep and compare with `dvc.lock`.

3 Skip or run

All hashes match and outputs exist? Skip. Otherwise run `cmd`.

4 Cascade

A stage that ran produces new output hashes, so every downstream stage is now out of date too.

5 Rewrite the lock

New hashes are written, and outputs are saved into the cache.



3. Edit one comment in `src/train.py` and run `dvc status`. Only `train` is out of date.

4. Now edit `src/prepare.py` instead and run `dvc status` again. **both** stages are affected.

5. Run `dvc dag` and check the picture matches your expectation.

Expected result – a change to a late stage affects only that stage, while a change to an early one cascades downstream. Watching that cascade is what turns "DVC caches things" into "DVC understands my pipeline".

Parameters: config DVC can see

Hard-coded hyperparameters make a stage un-reproducible: you changed `n_estimators` from 100 to 200, DVC saw no dependency change, and skipped the stage. `params` fixes that.

params.yaml

Copy

```
prepare:
  test_size: 0.2
  random_state: 42

train:
  n_estimators: 100
  max_depth: 8
  learning_rate: 0.05
```

dvc.yaml

Copy

```
stages:
  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/prepared.csv
    params:
      - train.n_estimators
```



- models/model.pkl

src/train.py

Copy

```
import yaml

with open("params.yaml") as f:
    params = yaml.safe_load(f)["train"]

model = RandomForestClassifier(
    n_estimators=params["n_estimators"],
    max_depth=params["max_depth"],
)
```

Now changing a number in `params.yaml` marks the stage out of date, as changing the code would. You can list individual keys as above, or `- train` to depend on the whole section.

BASH

Copy

```
dvc params diff          # what changed since the last commit
dvc params diff HEAD~3   # or against any revision
```

Parameters are the seam between "config" and "code"

Anything you might reasonably want to sweep belongs in `params.yaml` : learning rates, thresholds, feature flags, split ratios. Anything structural belongs in code. Getting this line in the right place is what makes experiment tracking pleasant later, because DVC can then vary parameters without touching a single source file.

► TRY IT

1. Move one hard-coded number from your training script into `params.yaml` and declare it in `dvc.yaml` .
2. Run `dvc repro` , then run it again and confirm the stage is skipped.



Expected result – DVC reporting the specific parameter that changed rather than a vague "modified". That precision in `dvc status` is what makes a long pipeline debuggable.

Metrics and plots: results that diff in Git

A pipeline that produces a model but no numbers is a pipeline you cannot reason about. DVC has two output kinds for exactly this.

dvc.yaml

Copy

```
stages:
  evaluate:
    cmd: python src/evaluate.py
    deps:
      - src/evaluate.py
      - models/model.pkl
      - data/test.csv
    metrics:
      - metrics.json:
        cache: false
    plots:
      - plots/roc.json:
        cache: false
        x: fpr
        y: tpr
      - plots/confusion.csv:
        cache: false
        template: confusion
        x: actual
        y: predicted
```

metrics.json

Copy



```
11 . 0.9104,  
  "auc": 0.9687,  
  "train_seconds": 42.7  
}
```

BASH

Copy

```
dvc metrics show          # the current numbers  
dvc metrics diff          # versus the last commit  
dvc metrics diff HEAD~5  # versus any revision  
dvc plots show           # render plots to an HTML file  
dvc plots diff HEAD~1    # overlay two revisions on one chart
```

Output kind	Tracked by	Use for
<code>outs</code>	DVC cache	Models, datasets, anything large or binary
<code>metrics</code> with <code>cache: false</code>	Git	Small JSON/YAML of scores
<code>plots</code> with <code>cache: false</code>	Git	Small CSV/JSON of curves and confusion matrices
<code>outs</code> with <code>cache: false</code>	Git	Any small text output you want diffable

Do not cache small text outputs

A 200-byte `metrics.json` in the DVC cache means comparing two commits requires a `dvc pull` from the remote. The same file in Git is instantly diffable, forever, by anyone with the repository. Reserve the cache for things Git is bad at.

► TRY IT



3. Run `dvc metrics diff` and read the before/after/change columns.

4. Add a plot output, a CSV of two columns is enough, and run `dvc plots diff HEAD~1`, then open the HTML it produces.

Expected result – a table showing how your numbers moved between two commits, and an overlaid chart of two revisions. That diff is the artifact you paste into a pull request, and it is the single most persuasive thing DVC produces.

Committing every attempt pollutes your history. `dvc exp` gives you a lightweight way to run many variations, compare them, and keep only the ones worth keeping.

BASH

Copy

```
# Run with a parameter override, without editing any file
dvc exp run --set-param train.n_estimators=300

# Queue several and run them together
dvc exp run --queue --set-param train.max_depth=4
dvc exp run --queue --set-param train.max_depth=8
dvc exp run --queue --set-param train.max_depth=16
dvc exp run --run-all --jobs 3

# Compare
dvc exp show
dvc exp show --only-changed          # hide columns that are identical everywhere
```

`dvc exp show` prints a table of every experiment with its parameters and metrics side by side, which is the fastest way to see that `max_depth=8` beat both neighbours.

TEXT

Copy



workspace	0.9231	0.9104	8	100
main	0.9105	0.8977	6	100
└─ exp-a1b2c	0.9402	0.9311	8	300
└─ exp-d4e5f	0.9188	0.9042	4	100
└─ exp-g7h8i	0.9377	0.9265	16	100

BASH

Copy

```
dvc exp apply exp-a1b2c      # bring it into your workspace
git add . && git commit -m "n_estimators=300: accuracy 0.940"

dvc exp branch exp-a1b2c my-branch  # or put it on its own branch
dvc exp remove --queued             # clean up
dvc exp gc --workspace              # garbage-collect experiments not referenced
```

✓ An experiment

- No commit needed, no branch needed
- Cheap to create and to throw away
- Comparable in one table
- Promote with `apply` or `branch`

✗ A commit per attempt

- Pollutes history with dead ends
- Comparing means reading log messages
- Rebasing later becomes unpleasant
- Nobody can find the good one

Experiments still require a clean-ish workspace

`dvc exp run` works from your current state, so uncommitted code changes are included, which is usually what you want while iterating. But it means two experiments run at different times can differ by code you forgot about. Commit the code, vary the parameters.

► TRY IT

1. Queue three experiments varying one parameter, then run them all with `--run-all`.



7. Commit, then run `dvc exp remove queued` and `dvc exp show` again.

Expected result – three runs compared in one table, one promoted into a real commit, and the rest discarded without leaving a trace in your history. That workflow is why you should stop committing every attempt.

Reading another repo's data: `get` versus `import`

Two commands let you use a DVC repository as a data source, which is how datasets get shared across projects.

BASH

Copy

```
# Look at what a repo tracks, without cloning it
dvc list https://github.com/iterative/dataset-registry get-started

# Download a file from it into the current directory
dvc get https://github.com/iterative/dataset-registry get-started/data.xml

# Download AND record where it came from
dvc import https://github.com/iterative/dataset-registry get-started/data.xml
```



```
dvc import --rev v1.2.0 https://github.com/org/data-repo datasets/train.parquet
dvc update datasets/train.parquet.dvc # pull the newer upstream version
```

`dvc import` writes a pointer that names the source repository, path, and revision. That means your project can state "we depend on version v1.2.0 of the shared training set" as a committed fact, and updating is a deliberate command rather than a silent drift.

A "data registry" is just a DVC repo with no code

Many teams keep one repository whose only job is to track canonical datasets.

Consumers `dvc import` from it. There is nothing special about it. It is the pattern that emerges naturally once two projects need the same data, and Senior level covers running one properly.

► TRY IT

1. Run `dvc list https://github.com/iterative/dataset-registry` and explore what is there.
2. `dvc get` a file from it and note that `git status` shows an untracked file with no pointer.
3. Delete it, then `dvc import` the same file and read the `.dvc` file it creates.
4. Find the `repo:` section, and the `rev_lock` that pins the exact upstream commit.

Expected result — `get` leaves you with an anonymous file, while `import` leaves you with a versioned dependency that names its source and revision. That distinction is the difference between copying data and depending on it.

Garbage collection: reclaiming the cache safely

The cache accumulates every version of everything you have ever added. That is the feature; it is also why your disk fills up.



```
dvc gc --all-tags          # keep what every tag needs
dvc gc --all-commits       # keep what every commit needs – the safest
dvc gc --cloud             # also clean the remote (dangerous, read below)
dvc gc --dry-run --workspace # show what would go, delete nothing
```

The flags are all about **which revisions count as "in use"**. The default, no flag at all, is `--workspace`, which is the most aggressive: anything not needed by the files currently checked out is deleted.

`dvc gc` deletes data, and `--cloud` deletes shared data

`dvc gc --workspace` on a repository with ten branches will delete the cached data for nine of them. Add `--cloud` and you have done it to your team's remote as well, where it is not recoverable. Always run with `--dry-run` first, and prefer `--all-commits` unless you have a specific reason to be aggressive.

► TRY IT

1. Create three versions of a dataset, committing each, so the cache has three entries.
2. Run `du -sh .dvc/cache` and note the size.



Expected result – `--workspace` proposes deleting the two older versions; `--all-commits` proposes deleting nothing, because every version is still reachable from a commit. Understanding that difference before you run it for real is the point of this exercise.

Reading a broken pipeline, in order

Debugging DVC has an order, and following it beats guessing.

01 Run `dvc status` first

It names the stage, the specific dependency, and the reason. Most problems are answered here and you can stop.

02 Then `dvc status -c`

Compares your cache against the remote. This is the answer to "my colleague cannot pull my data". You never pushed it.

03 Check the graph with `dvc dag`

A stage that never reruns often has a missing `deps` entry; a cycle error means two stages each claim the other's output.

04 Run the `cmd` by hand

Copy the command out of `dvc.yaml` and run it in your shell. If it fails there, DVC is innocent and you are debugging your script.

05 Force a rerun with `dvc repro -f`

Ignores the lock and reruns everything. If forcing fixes it, your `deps` are incomplete, and the real fix is declaring the missing dependency.



contacted and why a transfer failed.

Four failures cover most of what you will hit at this level, and each has a one-line cause:

Symptom	Cause	Fix
<code>ERROR: output 'x' is already tracked</code>	You <code>dvc add</code> ed a pipeline output	Remove the <code>.dvc</code> file; <code>outs</code> already tracks it
Stage never reruns after a code change	The script is not in <code>deps</code>	Add it
<code>dvc pull</code> says "missing cache files"	The data was never pushed	<code>dvc push</code> from wherever it was produced
Data is the wrong version after <code>git checkout</code>	You skipped <code>dvc checkout</code>	Run it, or <code>dvc install</code> the hooks

BASH

Copy

```
dvc status          # local truth
dvc status -c       # remote truth
dvc dag             # the graph as DVC sees it
dvc repro -f        # rebuild everything, ignoring the lock
dvc repro --dry      # show what would run, run nothing
dvc repro -s train   # just one stage, and its upstream
dvc repro -s train --single-item # exactly one stage, no upstream
```

► TRY IT: CAUSE EACH FAILURE ON PURPOSE

1. Remove a script from a stage's `deps` , change the script, and confirm `dvc repro` skips the stage.
2. Try to `dvc add` a pipeline output and read the exact error text.
3. Delete your local cache and try `dvc checkout` without having pushed. Read the error, then `dvc pull` .
4. For each one, get to the answer with `dvc status` rather than by remembering what you broke.



every input in `deps` matters more than any other table at this level.

Putting it all together

Everything above in one project. Nothing here is new. Read it as a whole and you should be able to justify every line.

project layout

Copy

```
.
├── .dvc/
│   ├── config                # remote URL - committed
│   └── config.local          # credentials - gitignored
├── .dvcignore
├── data/
│   ├── .gitignore            # written by DVC
│   ├── raw.csv.dvc           # pointer, committed
│   └── raw.csv                # the bytes, gitignored
├── src/
│   ├── prepare.py
│   ├── train.py
│   └── evaluate.py
├── models/
│   └── .gitignore             # written by DVC
├── params.yaml               # committed
├── metrics.json              # committed (cache: false)
├── plots/roc.csv             # committed (cache: false)
├── dvc.yaml                  # committed
└── dvc.lock                  # committed
```



dvc.yaml

Copy

```
stages:
  prepare:
    cmd: python src/prepare.py
    deps:
      - src/prepare.py
      - data/raw.csv          # the dvc add-tracked input
    params:
      - prepare.test_size
      - prepare.random_state
    outs:
      - data/train.csv        # cached: large, binary-ish, regenerable
      - data/test.csv

  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/train.csv        # produced upstream; the link in the graph
    params:
      - train.n_estimators
      - train.max_depth
    outs:
      - models/model.pkl      # cached: the artifact

  evaluate:
    cmd: python src/evaluate.py
    deps:
      - src/evaluate.py
      - models/model.pkl
      - data/test.csv
    metrics:
      - metrics.json:
          cache: false        # small text → Git, so diffs need no pull
    plots:
      - plots/roc.csv:
          cache: false
```



BASH

Copy

```
# The everyday loop
dvc repro                                # run only what is out of date
dvc metrics diff                         # did the numbers move?
git add dvc.lock metrics.json plots/ params.yaml
git commit -m "Deeper trees: accuracy 0.921 → 0.940"
dvc push                                # share the data and the model
git push
```

BASH

Copy

```
# What a colleague runs
git clone <repo> && cd <repo>
dvc pull                                # exactly your data and model
dvc repro                                # "Everything is up to date" – reproduced
```

Ten decisions in there are the whole lesson of this page:

Decision	Section
Data in a remote, pointers in Git	The problem DVC solves
<code>.dvc/config</code> committed, <code>config.local</code> not	Local versus committed configuration
<code>dvc add</code> for inputs you did not generate	Your first tracked dataset
<code>outs</code> : never <code>dvc add</code> : for generated files	Pipelines
Every script listed in <code>deps</code>	<code>dvc.lock</code> and how DVC knows
<code>dvc.lock</code> committed, never edited	<code>dvc.lock</code> and how DVC knows
Hyperparameters in <code>params.yaml</code>	Parameters
<code>cache: false</code> on metrics and plots	Metrics and plots



`dvc push` before `git push`

Remote storage

`dvc gc --dry-run` before any cleanup

Garbage collection

► TRY IT: THE ONE THAT MATTERS

1. Take this structure into a project you work on, adapting the stages to your own scripts.
2. Get `dvc repro` to a state where a second run reports everything up to date.
3. Push to a real remote, then clone the repository into a different directory, `dvc pull`, and `dvc repro`. Confirm it reports up to date rather than rebuilding.
4. Change one parameter, repro, and produce a `dvc metrics diff` you would be happy to paste into a pull request.

Expected result – a fresh clone that reproduces your result without rerunning anything, which is the strongest possible evidence that the pipeline is reproducible. Do this one even if you skip every other exercise.

What you can now do, and what comes next

You can version datasets and models with Git-sized pointers, restore any past version, share data through remote storage, express your workflow as a dependency graph that reruns only what changed, parameterise it, record metrics and plots that diff across commits, run and compare experiments without polluting history, depend on datasets from other repositories, keep your disk under control, and debug a pipeline methodically. That is a working practitioner's toolkit, enough to own the data and reproducibility story on a real project.

Can you...	
Explain what is in a <code>.dvc</code> file?	A hash, a size, and a path
Say why Git alone is wrong for data?	Full history of large binaries, forever



Name the two commands to travel back in time?	<code>git checkout</code> then <code>dvc checkout</code>
Say where credentials belong?	The environment or <code>.dvc/config.local</code>
Explain how <code>dvc repro</code> decides what to run?	Hashes in <code>dvc.lock</code> versus current inputs
Say why you never <code>dvc add</code> a stage output?	<code>outs</code> already tracks it
Explain why parameters need declaring?	Otherwise a config change is invisible
Say why metrics use <code>cache: false</code> ?	Small text belongs in Git, so diffs need no pull
Give the difference between <code>get</code> and <code>import</code> ?	Anonymous copy versus versioned dependency
Say what <code>dvc gc --workspace</code> deletes?	Everything not needed by the current checkout
Name the first command when something is wrong?	<code>dvc status</code> , then <code>dvc status -c</code>

Mid-level takes every one of those topics further: the hashing and link internals that explain performance, `dvc.yaml` templating with `foreach` and `matrix` , output modifiers like `persist` and `cache: false` , external and cloud-versioned outputs, `dvc import-url` , multiple and per-role remotes, experiment queues and cloud-based experiments, the CI patterns that make `dvc repro` a pull-request check, DVCLive for in-training logging, and monorepo layouts.

Senior then covers what you own when data versioning is your responsibility: the trust and access model across remotes, credentials that are never stored, immutability and retention on the storage side, cost and lifecycle policy, hashing at terabyte scale, data registries as a product, lineage and audit for regulated work, GDPR deletion against an immutable cache, CI/CD for data with self-hosted runners, and where DVC stops and a feature store or lakehouse begins.

NEXT UP

Beginner · Interview Review





MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.